Module 6: LLMOps I - Inference Optimization

Production AI: An Engineering Guide

Theja Tulabandhula

University of Illinois Chicago

Overview

LLM Foundations

Inference Optimization Techniques

Server Implementation

Performance & Governance

Operational Concerns

LLM Economics

Advanced Serving Patterns

Hands-On Demos

Real-World Case Studies

Lab & Assessments

Deployment and Future Trends

Key Takeaways

Overview

Module Overview

How does ChatGPT serve millions of concurrent requests while keeping response times under a second?

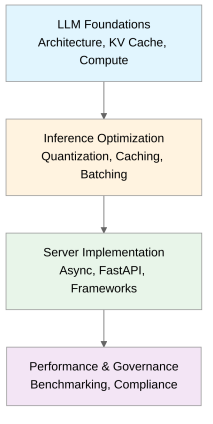
- This module covers the fundamentals of deploying and optimizing Large Language Model (LLM) inference systems
- Learn transformer architecture concepts, GPU memory and compute requirements
- Master quantization techniques, caching and batching strategies
- Understand performance benchmarking techniques for LLM serving

What You'll Build

By the end of this module, you will have:

1. A **FastAPI-based LLM inference server** with async request handling
2. An **inference caching layer** that reduces redundant computation
3. A **dynamic batching system** that maximizes GPU utilization
4. A **benchmark suite** measuring TTFT, TPOT, and throughput

Module Roadmap



LLM Foundations

LLM Architecture Overview

What is an LLM?

- **Large Language Model**: Neural network trained on massive text corpora
- Billions of parameters (GPT-3: 175B, LLaMA 2: 7B-70B)
- Built on **Transformer architecture** ("Attention Is All You Need", 2017)

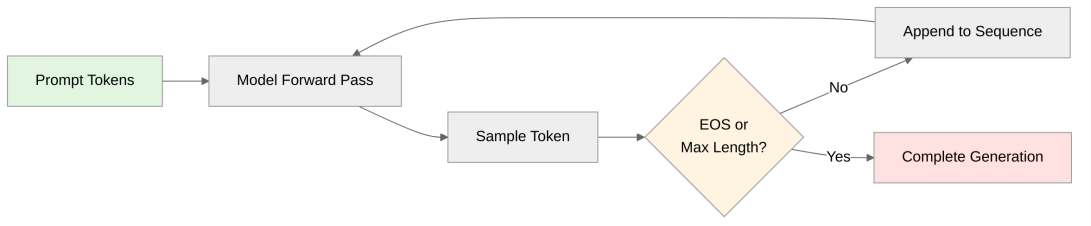
Three architectural flavors:

Type	Examples	Use Case
Encoder-only	BERT	Classification, understanding
Decoder-only	GPT, LLaMA	Text generation
Encoder-decoder	T5, BART	Translation, summarization

See: *LLM Architecture Overview*

How LLMs Generate Text

Autoregressive decoding: Predict one token at a time, feed it back as input



See: *LLM Architecture Overview*

Two Phases of LLM Inference

Phase	Description	Compute Characteristic
Prefill	Process all prompt tokens in parallel	Compute-bound (matrix multiplications)
Decode	Generate tokens one at a time	Memory-bound (loading weights repeatedly)

Key insight: Prefill benefits from GPU parallelism; decode is bottlenecked by memory bandwidth.

See: *LLM Architecture Overview*

Why LLM Inference is Memory-Bound

Traditional ML vs LLM inference:

Characteristic	Traditional ML	LLM Inference
Bottleneck	Compute (matrix multiplication)	Memory bandwidth
Optimization	Faster GPU (TFLOPS)	More memory bandwidth (TB/s)
Scaling	Batch size increases throughput	Batch size limited by memory

Why LLM Inference is Memory-Bound (cont.)

The memory bandwidth problem:

```
# 7B model in FP16 (using decimal GB for simplicity)
model_weights_gb = 7_000_000_000 * 2 / 1e9  # = 14 GB

# Each token generation requires loading ALL weights
# A100 40GB: 1.5 TB/s bandwidth
memory_bandwidth_tbs = 1.5  # TB/s

time_per_token_ms = (model_weights_gb / memory_bandwidth_tbs) * 1000 / 1000  #
↪    9.3 ms
tokens_per_second = 1000 / time_per_token_ms  #   107 tokens/sec
```

See: *LLM Compute Requirements*

Memory Bandwidth vs Compute

Why memory bandwidth matters more than compute:

```
GPU specifications:
A100: 312 TFLOPS (compute) vs 1.5 TB/s (memory)
H100: 989 TFLOPS (compute) vs 3.35 TB/s (memory)

Speedup potential:
Compute: 3.2x faster
Memory: 2.2x faster ← This is the bottleneck!
```

Implication for optimization:

- **Quantization** (reduce weight size → less data to load)
- **Batching** (amortize weight loading across requests)
- **Faster GPUs** (helps less than expected unless bandwidth improves)

Key insight: Choose GPUs for memory bandwidth (TB/s), not just compute (TFLOPS)

See: *LLM Compute Requirements*

Prefill Phase Characteristics

LLM inference has two distinct phases with different characteristics:

Prefill Phase (Process Prompt)

```
# Process all prompt tokens in parallel
prompt = "Explain transformers in detail"  # 5 tokens
# All 5 tokens processed simultaneously
# Heavy matrix multiplications (compute-bound)
```

Prefill Phase Characteristics (cont.)

Metric	Prefill
Processing	Parallel (all prompt tokens)
Bottleneck	Compute-bound
GPU Utilization	High (80-95%)
Batching Benefit	Excellent (process multiple prompts)
Typical Time	100ms for 512 tokens

See: *LLM Architecture Overview*

Decode Phase Characteristics

Decode Phase (Generate Response)

```
# Generate tokens one at a time
# Token 1: "Trans" ← depends on prompt
# Token 2: "formers" ← depends on token 1
# Token 3: "are" ← depends on tokens 1-2
# ... sequential dependency!
```

Metric	Decode
Processing	Sequential (one token at a time)
Bottleneck	Memory-bound (loading weights)
GPU Utilization	Low (20-40% without batching)
Batching Benefit	Critical (amortize weight loads)
Typical Time	20ms per token

Decode Phase: Latency Impact

Real-world example:

```
User query: "Write a 200-word essay on AI" (10 prompt tokens)

Prefill: 10 tokens × parallel = 50ms total
Decode: 200 tokens × 20ms = 4000ms total

Total latency: 4050ms
Decode phase: 98.8% of time!
```

Why this matters:

- **Optimization focus:** Decode phase dominates user-perceived latency
- **Batching strategy:** Critical for decode, helpful for prefill
- **Hardware choice:** Memory bandwidth more important than compute

See: *LLM Architecture Overview*

Why Can't We Parallelize Token Generation?

Autoregressive dependency chain:

Each token depends on ALL previous tokens:

```
# Cannot parallelize this!
token_1 = model(prompt)
token_2 = model(prompt + token_1)  # Needs token_1
token_3 = model(prompt + token_1 + token_2)  # Needs token_1 & token_2
...
```

Why Can't We Parallelize Token Generation? (cont.)

Implication for serving:

```
Can't parallelize within one request
↓
Must batch ACROSS requests to utilize GPU
↓
Dynamic batching becomes critical
```

See: *LLM Architecture Overview*

Training vs Inference: Parallelization

Comparison with training:

Phase	Parallelization	Why
Training	Full sequence parallel	All target tokens known upfront
Inference	Must be sequential	Future tokens unknown

Training vs Inference: Parallelization (cont.)

Why serving differs from training:

Training:

```
# All tokens available, process in parallel
loss = model(input_tokens, target_tokens)
# Batch size 1024, sequence length 2048
# All 2M tokens processed simultaneously
```

Training vs Inference: Parallelization (cont.)

Inference:

```
# Generate one token at a time
for i in range(max_tokens):
    next_token = model(generated_so_far)
    generated_so_far.append(next_token)
# Inherently sequential within request
```

See: *LLM Architecture Overview*

Future Directions for Parallelization

Future directions (emerging):

- **Speculative decoding:** Draft model predicts multiple tokens, target verifies
- **Parallel sampling:** Generate multiple candidates, select best
- **Still fundamentally constrained** by autoregressive nature

Key takeaway: The sequential nature of LLM generation is inherent to the architecture, making batching across requests the primary optimization lever

See: *LLM Architecture Overview*

Transformer Attention and KV Cache

Self-Attention Mechanism

Each token uses three learned projections:

- **Query (Q)**: "What am I looking for?"
- **Key (K)**: "What do I contain?"
- **Value (V)**: "What information do I provide?"

```
Attention(Q, K, V) = softmax(Q × K^T / √d_k) × V
```

Multi-head attention: 32 heads = 32 different types of relationships learned simultaneously

See: *Attention & KV Cache*

The KV Cache Optimization

Problem: Recomputing K and V for all previous tokens at every step is expensive

Solution: Store computed K/V values to avoid redundant work

Approach	At token 100
Without KV cache	Recompute K and V for all 99 previous tokens
With KV cache	Only compute K and V for the new token

Transforms generation from $O(n^2)$ to $O(n)$ complexity per layer

See: *Attention & KV Cache*

KV Cache Memory Impact

Component	Memory (FP16)
Model weights (7B)	~14 GB
KV cache per request (4K context)	~1 GB
8 concurrent requests	~8 GB additional

Critical for batching: Each request maintains its own KV cache!

See: *Attention & KV Cache*

LLM Compute Requirements

GPU Memory Components

Model Size	FP32	FP16	INT8	INT4
7B params	28 GB	14 GB	7 GB	3.5 GB
13B params	52 GB	26 GB	13 GB	6.5 GB
70B params	280 GB	140 GB	70 GB	35 GB

Rule of thumb: Memory (GB) Parameters (B) $\times$ Bytes per parameter

See: *LLM Compute Requirements*

Memory Bandwidth Bottleneck

During **decode phase**, each token requires loading entire model weights

GPU	Memory Bandwidth	Approximate Decode Throughput
A100 40GB	1.5 TB/s	~100 tokens/sec (7B FP16)
A100 80GB	2.0 TB/s	~130 tokens/sec (7B FP16)
H100	3.35 TB/s	~220 tokens/sec (7B FP16)

Key insight: Batching amortizes weight loading across multiple requests

See: *LLM Compute Requirements*

Calculating Maximum Batch Size

```python
def max_batch_size(
    gpu_memory_gb: float,
    model_memory_gb: float,
    kv_cache_per_request_gb: float,
    safety_margin: float = 0.85
) -> int:
    available = (gpu_memory_gb * safety_margin) - model_memory_gb
    return int(available / kv_cache_per_request_gb)

# Example: A100 40GB with 7B model
max_batch = max_batch_size(40, 14, 1.0)  # ~20 requests
```

See: *LLM Compute Requirements*

Inference Optimization Techniques

Quantization Fundamentals

What is Quantization?

Reduce numerical precision of model weights: FP32/FP16 $\rightarrow$ INT8/INT4

Precision	Memory for 7B Model	Relative Size
FP32	28 GB	100%
FP16/BF16	14 GB	50%
INT8	7 GB	25%
INT4	3.5 GB	12.5%

Trade-off: Smaller memory footprint vs. potential quality degradation

See: *Quantization Fundamentals*

Loading Quantized Models

```python
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
import torch

# 4-bit quantization with BitsAndBytes
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",  # Normalized float 4-bit
    bnb_4bit_compute_dtype=torch.float16
)
# see next slide
```

Loading Quantized Models (cont.)

```python
# continued from before

model_4bit = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-hf",
    quantization_config=bnb_config,
    device_map="auto"
)
```

See: *Quantization Fundamentals*

Quality vs. Speed Trade-offs

Task Type	INT8 Quality	INT4 Quality
Simple Q&A	~98% of FP16	~95% of FP16
Code generation	~95% of FP16	~85% of FP16
Math reasoning	~90% of FP16	~75% of FP16
Creative writing	~97% of FP16	~92% of FP16

Key insight: Tasks requiring precise reasoning degrade more than tasks with acceptable variation

See: *Quantization Fundamentals*

Quantization Precision: FP8

Operational considerations for choosing precision:

FP8 (8-bit floating point)

```
# Memory: 2x reduction from FP16
model_size = 7B * 1 byte = 7 GB (vs 14 GB FP16)

# Quality: <2% degradation
accuracy_loss = 0.5-1.5%  # Barely noticeable

# Speed: 1.5-2x faster inference
# Hardware: Requires H100 or newer
```

Use case: Production default for most applications

Quantization Precision: INT8 and INT4

INT8 (8-bit integer)

```
# Memory: 2x reduction from FP16
model_size = 7B * 1 byte = 7 GB

# Quality: ~5% degradation
accuracy_loss = 3-7%  # Noticeable on complex tasks

# Speed: 2-3x faster with INT8 support
# Hardware: Supported on most GPUs
```

Use case: Cost-sensitive applications, simple tasks

Quantization Precision: INT8 and INT4 (cont.)

INT4 (4-bit integer)

```
# Memory: 4x reduction from FP16
model_size = 7B * 0.5 byte = 3.5 GB

# Quality: 10-15% degradation
accuracy_loss = 8-15%  # Significant on reasoning

# Speed: 3-4x faster
# Hardware: Limited support, CPU-friendly
```

Use case: Edge deployment, mobile devices

See: *Quantization Fundamentals*

Production Quantization Deployment

Production deployment checklist:

1. **Calibration dataset required**

```
# Need representative data for quantization
calibration_data = [
    "Example prompt 1",
    "Example prompt 2",
    # ... 100-1000 examples
]

quantized_model = quantize(model, calibration_data, precision="int8")
```

Production Quantization Deployment (cont.)

2. A/B testing before rollout

```
# Split traffic to test quality impact
if user_id % 100 < 50:
    response = fp16_model.generate(prompt)
else:
    response = int8_model.generate(prompt)

# Measure: User satisfaction, task success, quality metrics
```

Production Quantization Deployment (cont.)

3. Task-specific quality validation

Task Type	INT8 Safe?	INT4 Safe?	Notes
Simple Q&A	Yes	Maybe	Low complexity
Code generation	Yes	No	Precision matters
Math reasoning	Careful	No	Significant degradation
Creative writing	Yes	Yes	Acceptable variation
Summarization	Yes	Careful	Monitor quality

See: *Quantization Fundamentals*

Quantization Cost-Benefit Analysis

Cost-benefit analysis:

```
# Baseline: FP16 on A100
gpu_cost_per_hour = 4.50
throughput_tokens_per_sec = 100
cost_per_million_tokens = (gpu_cost_per_hour / 3600) * (1_000_000 /
↪  throughput_tokens_per_sec) # Result: $12.50 per million tokens
```

Quantization Cost-Benefit Analysis (cont.)

```
# INT8 quantization
throughput_tokens_per_sec_int8 = 250  # 2.5x faster
cost_per_million_tokens_int8 = (gpu_cost_per_hour / 3600) * (1_000_000 /
↪  throughput_tokens_per_sec_int8) # Result: $5.00 per million tokens

# Savings: 60% cost reduction
# Risk: 3-5% quality degradation
# Decision: Worth it for most production use cases
```

See: *Quantization Fundamentals*

Inference Caching Strategies

Why Cache LLM Responses?

Scenario	Latency	Cost per Request
Cold cache (model inference)	500-2000ms	High (GPU time)
Warm cache (Redis lookup)	1-5ms	Negligible

Production cache hit rates of 20-40% can reduce infrastructure costs by 30-50%

See: *Inference Caching*

Cache Key Design

```python
import hashlib
import json

def make_cache_key(
    prompt: str, model: str, temperature: float, max_tokens: int
) -> str:
    """Create deterministic cache key from request parameters."""
    key_data = {
        "prompt": prompt.strip().lower(),
        "model": model,
        "temperature": temperature,
        "max_tokens": max_tokens
    }
    # see next slide
```

Cache Key Design

```python
    # continued from before
    content = json.dumps(key_data, sort_keys=True)
    return f"llm:{hashlib.sha256(content.encode()).hexdigest()}"
```

Important: Never include user IDs in cache keys!

See: *Inference Caching*

Caching Best Practices

Consideration	Recommendation
Determinism	Only cache when temperature=0
Privacy	Hash prompts, never store plaintext
Eviction	TTL-based (1 hour default)
Key collisions	Use SHA-256 for minimal risk

See: *Inference Caching*

When to Cache vs When Not To

Decision matrix for LLM response caching:

Caching WORKS well for:

Use Case	Cache Hit Rate	Why
FAQ systems	40-60%	Repeated questions
Code completion	30-50%	Common patterns
Template responses	50-70%	Fixed formats
Documentation queries	35-55%	Popular topics

When to Cache vs When Not To (cont.)

Caching FAILS for:

Use Case	Cache Hit Rate	Why
Creative writing	$<5\%$	High diversity
Personalized content	$<10\%$	User-specific
Real-time data	0%	Always changing
Conversational AI	5-15%	Context-dependent

See: *Inference Caching*

Caching Strategies

Caching strategies:

1. Exact Match (Simple)

```
cache_key = hash(prompt + model + temperature + max_tokens)
# Pro: Fast, simple
# Con: Misses similar prompts
```

2. Semantic Caching (Advanced)

```
embedding = embed(prompt)
similar = find_similar_embeddings(embedding, threshold=0.95)
if similar:
    return cached_response[similar]
# Pro: Handles similar prompts
# Con: Embedding overhead, false positives
```

Caching Strategies (cont.)

Cache invalidation strategies:

```python
# TTL-based (most common)
cache.set(key, value, ttl=3600)  # 1 hour expiry

# Version-based (model updates)
cache_key = f"{hash(prompt)}:v{model_version}"

# Manual invalidation (user request)
if user.opts_out_of_cache:
    bypass_cache = True
```

Cache Privacy and Bypass Rules

Privacy considerations:

```
#  WRONG: Cache plaintext prompts
cache_key = prompt  # Can leak user data!

#  CORRECT: Hash prompts
cache_key = hashlib.sha256(prompt.encode()).hexdigest()

#  BETTER: Never share cache across users
cache_key = f"{user_id}:{hash(prompt)}"
```

Cache Privacy and Bypass Rules (cont.)

When to bypass cache:

- User explicitly requests fresh response
- Real-time data needed (news, stock prices)
- Temperature > 0 (non-deterministic)
- Sensitive/private prompts

See: *Inference Caching*

Decoding Strategies Impact Latency

Strategy	Speed	Quality	When to Use
Greedy	Fastest	Deterministic	Factual Q&A
Temperature	Fast	Adjustable creativity	Creative writing
Top-k	Medium	Diverse	Brainstorming
Top-p	Medium	Balanced	General chat

Trade-off: Sampling adds latency but improves diversity

For Milestone 5: Your API should expose `temperature` parameter!

See: *Decoding Strategies*

Tokenization Affects Cache Keys

Example: "Hello, world!" with BPE

```
tokens = ["Hello", ",", " world", "!"]
token_ids = [15496, 11, 995, 0]
num_tokens = 4  # Used for billing!
```

Why this matters for Milestone 5:

- Cache keys must include tokenization
- Token count affects API billing
- Different models = different tokenizers

Common pitfall: Caching "Hello world" "Hello world" (extra space)

See: *Tokenization (BPE)*

Dynamic Request Batching

Why Batching Matters

LLM inference is **memory-bandwidth bound** during token generation

Scenario	Weight Loads	Tokens Generated	Efficiency
1 request	1 load	1 token	1x
8 requests batched	1 load	8 tokens	8x

Without batching: ~100 tokens/sec

With batching: ~500+ tokens/sec aggregate

See: *Dynamic Request Batching*

Dynamic Batching Triggers

Two triggers balance throughput against latency:

- **Batch size threshold:** Process when N requests accumulate
- **Timeout:** Process whatever is available after T milliseconds

Parameter	Description	Suggested Value
`max_batch_size`	Maximum requests per batch	8
`max_wait_ms`	Timeout for partial batch	50ms
`max_sequence_length`	Cap on total tokens	2048

See: *Dynamic Request Batching*

Static vs Continuous Batching

The problem with static batching:

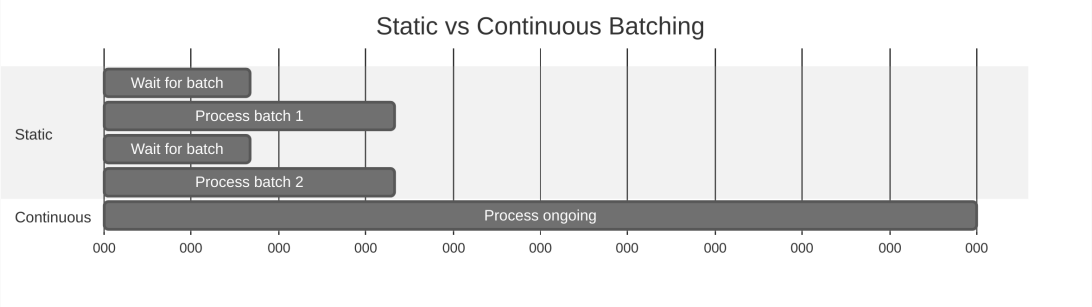
```
Static Batch:
[Wait for 8 requests] → [Process all 8] → [Wait for next 8]

Problems:
1. GPU idle while waiting for batch to fill
2. Fast requests wait for slow ones (stragglers)
3. Fixed batch size inefficient for variable lengths
```

Continuous batching solution:

```
Continuous Batch:
Request 1 arrives → Join batch
Request 2 arrives → Join batch
Request 1 completes → Leave batch, new request joins
Request 3 arrives → Join batch
...
```

Static vs Continuous Batching (cont.)

Key advantages:



See: *Dynamic Request Batching*

Continuous Batching Benefits

Timeline comparison:

Time	Static Batching	Continuous Batching
0ms	Waiting for requests	Processing req 1, 2
50ms	Batch full, start processing	Req 1 done, req 3 joins
150ms	Batch done, waiting	Req 2 done, req 4 joins
200ms	Next batch starts	Still processing...

Performance impact:

- **GPU utilization:** 40% $\rightarrow$ 85%
- **Throughput:** 2-3x improvement
- **Latency:** Lower variance (no straggler problem)

Continuous Batching Benefits (cont.)

Requirements for continuous batching:

1. **Dynamic memory management** (PagedAttention)
2. **Efficient KV cache allocation/deallocation**
3. **Request-level tracking** (not batch-level)

Implementation: vLLM, TensorRT-LLM support this natively

See: *Dynamic Request Batching*

PagedAttention: The Fragmentation Problem

The KV cache fragmentation problem:

```
Traditional allocation:
Request 1: [     ] 800 tokens used, 200 wasted
Request 2: [     ] 400 tokens used, 600 wasted
Request 3: [     ] 1000 tokens used, 0 wasted

Total memory: 3000 allocated, 2200 used
Waste: 27%
```

PagedAttention: The Fragmentation Problem (cont.)

PagedAttention solution: Treat KV cache like OS virtual memory

```
Paged allocation (block size = 16 tokens):
Request 1: [50 blocks] = 800 tokens, 0 waste
Request 2: [25 blocks] = 400 tokens, 0 waste
Request 3: [63 blocks] = 1000 tokens, 8 waste

Total memory: 138 blocks = 2208 allocated
Waste: <1%
```

PagedAttention: Key Concepts

Key concepts:

1. Fixed-size blocks

```
BLOCK_SIZE = 16  # tokens per block
block_size_bytes = BLOCK_SIZE * hidden_dim * 2 * sizeof(fp16)
                 = 16 * 4096 * 2 * 2 = 256 KB per block
```

2. Block tables (logical → physical mapping)

```
request_1_blocks = [0, 1, 2, 5, 8, 12, ...]  # Non-contiguous!
request_2_blocks = [3, 4, 6, 7, 9, ...]
```

PagedAttention: Key Concepts (cont.)

3. Dynamic allocation

```
def append_token(request_id):
    if request.current_block_full():
        new_block = allocate_free_block()
        request.blocks.append(new_block)
    write_kv_to_current_block(request_id, token)
```

See: *Dynamic Request Batching*

PagedAttention Benefits

Benefits:

Metric	Traditional	PagedAttention	Improvement
Memory waste	20-40%	$<4\%$	5-10x
Max concurrent requests	12	24	2x
Fragmentation	High	Near-zero	

PagedAttention Benefits (cont.)

Additional capability: Prefix sharing

```
# Multiple requests with same system prompt
system_prompt_blocks = [0, 1, 2, 3]  # Shared!
request_1_blocks = system_prompt_blocks + [4, 5, 6]
request_2_blocks = system_prompt_blocks + [7, 8, 9]

# Save 4 blocks per request
```

Result: 2-4x more concurrent requests on same GPU

See: *Dynamic Request Batching*

Batching Implementation Pattern

```python
async def submit(self, prompt: str, max_tokens: int) -> str:
    future = asyncio.Future()
    request = PendingRequest(prompt, max_tokens, future, time.time())

    async with self.lock:
        self.pending.append(request)
        should_process = len(self.pending) >= self.max_batch_size

    if should_process:
        asyncio.create_task(self._process_batch())

    return await future
```

See: *Dynamic Request Batching*

Batch Configuration Trade-offs

The batch configuration problem:

Batch size too small:

```
max_batch_size = 2
# GPU utilization: 30%
# Throughput: Poor (only 2 requests processed together)
# Latency: Good (requests don't wait long)
```

Batch size too large:

```
max_batch_size = 32
# GPU utilization: High risk of OOM
# Throughput: Could be good if memory allows
# Latency: Poor (long requests block entire batch)
```

Batch Configuration Trade-offs (cont.)

Timeout too short:

```
max_wait_ms = 10
# Batch sizes: Often 1-2 (didn't wait long enough)
# Latency: Excellent
# Throughput: Poor (underutilized GPU)
```

Timeout too long:

```
max_wait_ms = 200
# Batch sizes: Good
# Latency: Poor (requests wait too long)
# Throughput: Good
```

See: *Dynamic Request Batching*

Empirical Batch Tuning Process

Empirical tuning process:

```
# Step 1: Start conservative
config = {
    "max_batch_size": 4,
    "max_wait_ms": 50,
    "max_tokens": 200
}

# Step 2: Measure baseline
metrics = benchmark(config)
# avg_batch_size: 2.5
# p95_latency: 300ms
# gpu_util: 55%
```

Empirical Batch Tuning Process (cont.)

```
# Step 3: Increase batch size
config["max_batch_size"] = 8
metrics = benchmark(config)
# avg_batch_size: 4.2
# p95_latency: 350ms (+17%)
# gpu_util: 75% (+20%)

# Step 4: Tune timeout
config["max_wait_ms"] = 30
metrics = benchmark(config)
# avg_batch_size: 3.8
# p95_latency: 280ms (best!)
# gpu_util: 72%
```

Batch Tuning: Workload Configuration

Optimal configuration depends on workload:

Workload Type	Batch Size	Timeout	Rationale
Interactive chat	4-8	20-50ms	Latency sensitive
Batch processing	16-32	100-200ms	Throughput focused
Mixed traffic	8-16	50-100ms	Balance

Batch Tuning: Workload Configuration (cont.)

Load-dependent config (advanced):

```python
def get_batch_config(queue_depth: int) -> dict:
    if queue_depth < 5:
        # Light load: prioritize latency
        return {"max_batch_size": 4, "max_wait_ms": 20}
    elif queue_depth < 20:
        # Medium load: balance
        return {"max_batch_size": 8, "max_wait_ms": 50}
    else:
        # Heavy load: maximize throughput
        return {"max_batch_size": 16, "max_wait_ms": 100}
```

See: *Dynamic Request Batching*

Server Implementation

Python Async Fundamentals

Why Async for LLM Serving?

Approach	10 concurrent users	100 concurrent users
Synchronous	10 second wait	100 second wait
Asynchronous	~1 second each	~1-2 seconds each

LLM serving involves:

- Network I/O (HTTP requests)
- Database I/O (Redis cache)
- Multiple concurrent connections

See: *Python Async Fundamentals*

Async/Await Syntax

```python
# Sequential: 3 seconds total
async def sequential():
    await slow_operation("Task 1", 1.0)
    await slow_operation("Task 2", 1.0)
    await slow_operation("Task 3", 1.0)

# Concurrent: 1 second total!
async def concurrent():
    results = await asyncio.gather(
        slow_operation("Task 1", 1.0),
        slow_operation("Task 2", 1.0),
        slow_operation("Task 3", 1.0),
    )
```

See: *Python Async Fundamentals*

Common Async Mistakes

```python
# WRONG: Blocks the entire event loop
async def bad_function():
    time.sleep(5)  # Nothing else can run!

# CORRECT: Yields control while sleeping
async def good_function():
    await asyncio.sleep(5)  # Other coroutines run
```

```python
# WRONG: Missing await
data = fetch_data()  # Returns coroutine object!

# CORRECT: Await the coroutine
data = await fetch_data()  # Returns actual data
```

See: *Python Async Fundamentals*

Protecting Shared State with Locks

```python
class ThreadSafeBatcher:
    def __init__(self, max_batch_size: int = 8):
        self.pending: List[PendingRequest] = []
        self.lock = asyncio.Lock()  # Protects self.pending

    async def submit(self, prompt: str) -> str:
        async with self.lock:
            self.pending.append(request)
            should_process = len(self.pending) >= self.max_batch_size

        if should_process:
            asyncio.create_task(self._process_batch())
```

See: *Asyncio Concurrency Patterns*

Connection Limiting with Semaphores

```python
class RateLimitedClient:
    def __init__(self, max_concurrent: int = 10):
        self.semaphore = asyncio.Semaphore(max_concurrent)

    async def fetch(self, url: str) -> str:
        async with self.semaphore:  # Only 10 concurrent
            async with self.session.get(url) as response:
                return await response.text()
```

Use for: Redis connections, model inference calls, external services

See: *Asyncio Concurrency Patterns*

Background Tasks and Lifecycle

```python
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    await batcher.start_timeout_processor()
    yield
    # Shutdown
    await batcher.stop_timeout_processor()

app = FastAPI(lifespan=lifespan)
```

Always cancel background tasks on shutdown to prevent orphaned coroutines

See: *Asyncio Concurrency Patterns*

FastAPI LLM Server Structure

Recommended Project Organization

```
milestone5/
  src/
      server.py      # FastAPI entrypoint
      batching.py    # Dynamic batching logic
      caching.py     # Cache implementation
      inference.py   # Model loading/inference
      config.py      # Configuration management
      models.py      # Pydantic schemas
  benchmarks/
  tests/
  requirements.txt
  README.md
```

See: *FastAPI LLM Server Structure*

Configuration Management

```python
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    # Model settings
    model_name: str = "meta-llama/Llama-2-7b-hf"

    # Batching settings
    max_batch_size: int = 8
    batch_timeout_ms: float = 50.0

    # Caching settings
    redis_url: str = "redis://localhost:6379"
    cache_ttl_seconds: int = 3600
    # see next slide
```

Configuration Management

```
# continued from before
class Config:
    env_prefix = "LLM_"  # LLM_MAX_BATCH_SIZE=16
```

See: *FastAPI LLM Server Structure*

LLM Serving Frameworks Comparison

Framework	Key Feature	Best For	Complexity
vLLM	PagedAttention	High-throughput serving	Medium
TGI	Production-ready API	Quick deployment	Low
TensorRT-LLM	NVIDIA optimization	Maximum GPU efficiency	High
Ollama	Local development	Prototyping	Very Low

See: *LLM Serving Frameworks*

vLLM Quick Start

```
# Install vLLM
pip install vllm

# Start server with OpenAI-compatible API
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-2-7b-hf \
    --port 8000
```

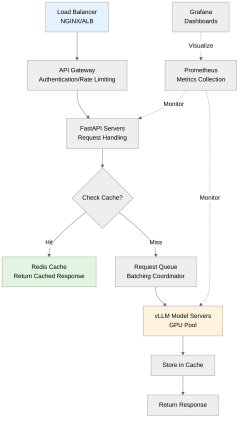
Key innovations:

- PagedAttention: Manages KV cache like virtual memory pages
- Continuous batching: Requests join/leave dynamically
- Near-100% GPU utilization

See: *LLM Serving Frameworks*

Production LLM Serving Stack

Real-world architecture for LLM serving:



Production LLM Serving Stack (cont.)

Component responsibilities:

Component	Purpose	Technology Choices
Load Balancer	Distribute traffic, health checks	NGINX, AWS ALB, GCP Load Balancer
API Gateway	Auth, rate limiting, routing	Kong, AWS API Gateway, custom FastAPI
FastAPI Servers	Business logic, request validation	FastAPI, async Python
Cache Layer	Store repeated responses	Redis, Memcached
Queue/Batcher	Accumulate requests for batching	Custom (asyncio), Celery
Model Servers	GPU inference	vLLM, TensorRT-LLM, TGI
Monitoring	Metrics, alerts,	Prometheus, Grafana, DataDog

Production LLM Serving Stack (cont.)

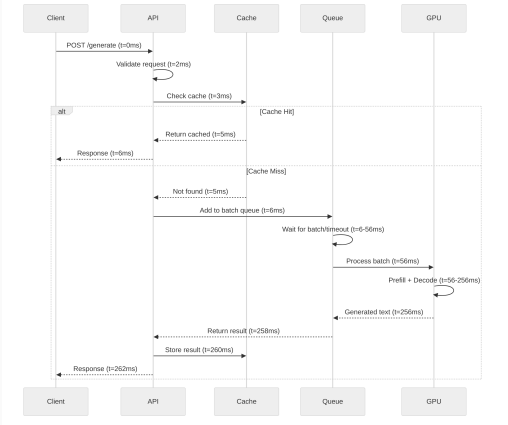
Why this architecture?

- **Horizontal scaling:** Add more FastAPI servers for higher RPS
- **Caching:** Reduce 20-40% of GPU load
- **Batching:** Maximize GPU utilization
- **Observability:** Understand system behavior

See: *FastAPI LLM Server Structure*

The Complete Request Lifecycle

Step-by-step flow with timing:



Request Lifecycle: Latency Breakdown

Typical latency breakdown:

Stage	Latency	Percentage	Optimization Target
Validation	1-2ms	<1%	-
Cache lookup	1-5ms	1-2%	Redis optimization
Queue wait	10-50ms	5-20%	Batch size/timeout tuning
GPU inference	100-500ms	60-90%	Quantization, batching
Cache write	1-2ms	<1%	-
Network	5-20ms	2-5%	Regional deployment

Request Lifecycle: Latency Breakdown (cont.)

Where to optimize:

1. **Cache hit rate:** 30% cache = 30% skip GPU entirely
2. **Batch waiting:** Tune `max_wait_ms` vs `max_batch_size`
3. **GPU inference:** Quantization, continuous batching, better hardware

Key insight: Most user-perceived latency is GPU inference time

See: *FastAPI LLM Server Structure*

Performance & Governance

LLM Benchmarking Metrics

Metric	Definition	Why It Matters
TTFT	Time to First Token	User-perceived responsiveness
TPOT	Time Per Output Token	Generation speed
Throughput	Tokens/second (aggregate)	System capacity
P50/P95/P99	Percentile latencies	Tail latency for SLOs
Cache Hit Rate	% from cache	Cost efficiency

See: *LLM Benchmarking*

Calculating LLM Metrics

```python
def calculate_tpot(metrics: RequestMetrics) -> float:
    """Time Per Output Token in milliseconds."""
    if metrics.output_tokens <= 1:
        return 0
    # Exclude TTFT from TPOT calculation
    decode_time = metrics.total_time_ms - metrics.ttft_ms
    return decode_time / (metrics.output_tokens - 1)
```

See: *LLM Benchmarking*

Benchmark Suite Design

1. **Single Request Baseline** - Measure without batching/caching
2. **Concurrent Load Test** - Measure batching effectiveness

```python
async def benchmark_concurrent(client, prompts, concurrency=10):
    semaphore = asyncio.Semaphore(concurrency)
    async def make_request(prompt):
        async with semaphore:
            return await measure_request(client, prompt)
    tasks = [make_request(p) for p in prompts]
    return await asyncio.gather(*tasks)
```

3. **Cache Warm-up Test** - Compare cold vs warm cache

See: *LLM Benchmarking*

Load Testing with Locust

```python
from locust import HttpUser, task, between

class LLMUser(HttpUser):
    wait_time = between(1, 3)

    @task
    def generate_text(self):
        self.client.post("/generate", json={
            "prompt": "Explain machine learning",
            "max_tokens": 100
        })
```

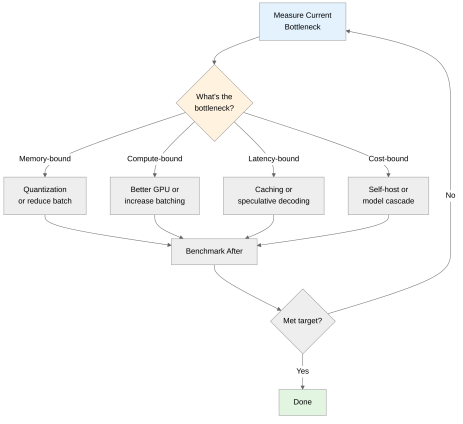
Load Testing with Locust (cont.)

```
locust -f benchmark.py --host http://localhost:8000 \
    --users 50 --spawn-rate 5 --run-time 60s
```

See: *LLM Benchmarking*

Optimization Decision Framework

Systematic approach to improving LLM serving performance:



Optimization Decision Framework (cont.)

Step 1: Measure baseline

```
# Collect metrics
baseline_metrics = {
    "p50_latency_ms": 250,
    "p95_latency_ms": 800,
    "throughput_tok_s": 120,
    "gpu_utilization": 0.45,  # 45%
    "cost_per_1m_tokens": 12.50  # $12.50
}
```

See: *LLM Benchmarking*

Identifying and Fixing Bottlenecks

Step 2: Identify bottleneck

Symptom	Likely Bottleneck	First Action
Low GPU util (<50%)	Insufficient batching	Increase batch size/timeout
High memory (>85%)	Memory capacity	Quantization or reduce batch
High latency, low throughput	Sequential processing	Implement batching
High cost per token	Inefficient serving	Self-host or optimize

Identifying and Fixing Bottlenecks (cont.)

Step 3: Choose optimization

- **Always benchmark before and after**
- **Change one variable at a time**
- **A/B test quality impact**

Step 4: Iterate

Most production systems need 3-5 iterations to reach target performance

See: *LLM Benchmarking*

LLM Serving Governance

Why Governance Matters

Risk Category	Example	Consequence
Privacy	PII cached and exposed	GDPR fines, trust loss
Security	Prompt injection attacks	Data leaks
Misuse	Harmful content generation	Legal liability
Compliance	Data residency violations	Regulatory penalties

See: *LLM Serving Governance*

Privacy-Safe Caching

```python
# PRIVACY RISK: Caching full prompts
cache_key = hash(prompt + model + str(temperature))

# MITIGATION: Hash prompts, never store plaintext
def privacy_safe_cache_key(prompt: str, params: dict) -> str:
    content = json.dumps({
        "prompt_hash": hashlib.sha256(prompt.encode()).hexdigest(),
        "params": params
    }, sort_keys=True)
    return f"llm:{hashlib.sha256(content.encode()).hexdigest()}"
```

See: *LLM Serving Governance*

Data Retention Policies

Data Type	Retention	Justification
Cache entries	1 hour	Balance hit rate vs. freshness
Request logs	30 days	Debugging and audit trail
Metrics	90 days	Performance trending

Consider implementing:

- Cache bypass option for users
- Cache invalidation API
- User opt-out for caching

See: *LLM Serving Governance*

Rate Limiting for Abuse Prevention

```python
class RateLimiter:
    def __init__(self, requests_per_minute: int = 60):
        self.rpm = requests_per_minute
        self.requests = defaultdict(list)

    # see next slide
```

Rate Limiting for Abuse Prevention (cont.)

```python
# continued from before
def check(self, user_id: str) -> bool:
    now = time.time()
    minute_ago = now - 60
    # Clean old requests
    self.requests[user_id] = [
        t for t in self.requests[user_id] if t > minute_ago
    ]
    if len(self.requests[user_id]) >= self.rpm:
        return False
    self.requests[user_id].append(now)
    return True
```

See: *LLM Serving Governance*

Operational Concerns

GPU Autoscaling Strategies

Challenge: GPU instances take 30-60 seconds to start

Why this matters:

- Sudden traffic spike → Users wait during scale-up
- Idle GPUs → Wasted money ($3-7/hour per GPU)
- Need balance between responsiveness and cost

Strategy 1: Predictive Scaling

```
# Scale based on time-of-day patterns
scaling_schedule = {
    "00:00-06:00": 2,  # Night: 2 GPUs
    "06:00-09:00": 5,  # Morning ramp: 5 GPUs
    "09:00-17:00": 10, # Business hours: 10 GPUs
    "17:00-22:00": 6,  # Evening: 6 GPUs
    "22:00-00:00": 3,  # Late night: 3 GPUs
}
```

GPU Autoscaling: Queue-Depth and Warm Pool

Strategy 2: Queue-Depth Triggered

```
# Scale based on request backlog
if queue_depth > 50:
    scale_up(num_gpus=2)
elif queue_depth < 10 and num_gpus > min_gpus:
    scale_down(num_gpus=1)
```

Strategy 3: Warm Pool

```
# Keep extra capacity ready
active_gpus = 8  # Serving requests
warm_pool = 2    # Ready but idle
cold_reserve = 5 # Can spin up if needed

# Instant response to spikes from warm pool
```

GPU Autoscaling: Queue-Depth and Warm Pool (cont.)

Recommended approach: Hybrid

- Predictive for known patterns
- Warm pool (1-2 GPUs) for bursts
- Queue-depth for unexpected load

See: *LLM Inference Economics*

Multi-GPU Patterns: Model & Pipeline Parallelism

When you need to scale beyond 1 GPU:

Pattern 1: Model Parallelism (Tensor Parallelism)

```
Split model layers across GPUs
Layer 1-12  → GPU 0
Layer 13-24 → GPU 1
Layer 25-36 → GPU 2
```

- **Use case:** Model too large for single GPU (70B+)
- **Latency:** Same as single GPU (sequential processing)
- **Throughput:** Same as single GPU

Multi-GPU Patterns: Model & Pipeline Parallelism (cont.)

Pattern 2: Pipeline Parallelism

```
Different pipeline stages on different GPUs
Prefill  → GPU 0
Decode 1 → GPU 1
Decode 2 → GPU 2
```

- **Use case:** Long generation sequences
- **Complexity:** High (managing pipeline)

See: *LLM Serving Frameworks*

Multi-GPU Patterns: Data Parallelism & Hybrid

Pattern 3: Data Parallelism (Replicas)

```
Full model copy on each GPU
Request 1 → GPU 0
Request 2 → GPU 1
Request 3 → GPU 2
```

- **Use case:** High request volume, model fits on one GPU
- **Throughput:** Linear scaling (3 GPUs = 3x throughput)
- **Simple load balancing**

Multi-GPU Patterns: Data Parallelism & Hybrid (cont.)

Pattern 4: Hybrid (Most Common)

```
2 model copies (data parallelism)
Each copy on 4 GPUs (tensor parallelism)
Total: 8 GPUs

Serves 2x requests of 70B model
```

Decision guide:

- Model $<$ 20 GB $\rightarrow$ Data parallelism
- Model $>$ 40 GB $\rightarrow$ Tensor parallelism required
- High traffic $\rightarrow$ Data parallelism + tensor parallelism

See: *LLM Serving Frameworks*

Production Failures: Out of Memory (OOM)

1. Out of Memory (OOM)

```
Error: CUDA out of memory
Cause: KV cache exhaustion
```

Symptoms:

- Sudden crash during batch processing
- Happens with long prompts or large batches

Production Failures: Out of Memory (OOM) (cont.)

Prevention:

```
MAX_BATCH_SIZE = 8
MAX_SEQUENCE_LENGTH = 2048

if batch_size > MAX_BATCH_SIZE:
    split_batch()

if prompt_length > MAX_SEQUENCE_LENGTH:
    truncate_or_reject()
```

Production Failures: Request Timeouts

2. Request Timeouts

```
Error: Request timeout after 30s
Cause: Batch processing taking too long
```

Symptoms:

- Some requests complete, others timeout
- Usually due to few very long requests

Prevention:

```python
REQUEST_TIMEOUT = 10  # seconds per request
MAX_OUTPUT_TOKENS = 200  # Cap generation length

if generation_time > REQUEST_TIMEOUT:
    early_stop()
```

See: *LLM Serving Frameworks*

Production Failures: Stragglers and Cold Starts

3. Straggler Problem

```
Batch with 8 requests
7 complete in 1s, 1 takes 10s
All 8 wait for slowest
```

Prevention:

- Use continuous batching (requests leave when done)
- Set per-request timeouts
- Monitor P95/P99 latencies

Production Failures: Stragglers and Cold Starts (cont.)

4. Cold Start Latency

```
First request after idle: 2-5 seconds
Subsequent requests: 100-500ms
```

Cause: Model loading, GPU warmup, cache initialization

Prevention:

```python
# Keep-alive: Periodic dummy requests
async def keep_alive():
    while True:
        await generate("test", max_tokens=1)
        await asyncio.sleep(60)  # Every minute
```

See: *LLM Serving Frameworks*

Critical Metrics for LLM Serving

1. GPU Metrics

```
gpu_utilization = gpu_active_time / total_time
# Target: >70% utilization
# <50% = underutilized, wasting money
# >95% = may be bottleneck

gpu_memory_used = current_memory / total_memory
# Target: 60-80% (leave room for spikes)
# >90% = OOM risk
```

Critical Metrics for LLM Serving (cont.)

2. Request Metrics

```
# Latency distribution
p50_latency  # Median user experience
p95_latency  # 95% of users
p99_latency  # Worst 1% (tail latency)

# Throughput
requests_per_second
tokens_per_second  # Aggregate generation rate
```

See: *LLM Benchmarking*

Batch, Cache & Alert Configuration

3. Batch Metrics

```
batch_size_distribution = percentiles([1, 2, 4, 8, 16])
# Underutilization if mostly size 1-2
# Good: Average 6-8

queue_depth = pending_requests.count()
# Spikes indicate capacity issues
```

Batch, Cache & Alert Configuration (cont.)

4. Cache Metrics

```
cache_hit_rate = cache_hits / total_requests
# Target: 20-40% for production
# <10% = caching not helpful
# >60% = very repetitive workload

cache_memory_used_gb
# Monitor growth over time
```

Dashboard Layout and Alert Thresholds

Dashboard layout recommendation:

```
[GPU Utilization] [GPU Memory] [Request Rate]
[P50/P95/P99 Latency Chart Over Time]
[Batch Size Distribution] [Queue Depth]
[Cache Hit Rate] [Error Rate]
```

Alert thresholds:

- GPU utilization $<30\%$ for >10 min $\rightarrow$ Scale down
- GPU memory $>85\%$ $\rightarrow$ OOM risk
- P95 latency >2x baseline $\rightarrow$ Investigate
- Error rate $>1\%$ $\rightarrow$ Page on-call

See: *LLM Benchmarking*

LLM Economics

Why LLM Inference is Expensive

Phase	Cost Type	Typical Cost
Training	One-time	$5M-$100M+
Inference	**Recurring**	**Every request costs GPU time**

Cost drivers:

- GPU time per token
- Memory bandwidth requirements
- Batching efficiency
- Model size

Key insight: Batching reduces $/token by amortizing GPU costs across requests

See: *LLM Inference Economics*

GPU Cost Breakdown by Provider

Cloud GPU pricing comparison (as of 2025):

GPU Type	AWS	GCP	Azure	On-Demand $/hr	Reserved $/hr	Spot $/hr
A100 40GB	p4d	a2-highgpu	ND A100 v4	$3.20	$2.10	$0.90
A100 80GB	p4de	a2-ultragpu	ND A100 v4	$4.50	$3.00	$1.30
H100 80GB	p5	a3-highgpu	-	$7.50	$5.00	$2.50

GPU Cost: Self-Hosting Economics

Cost per 1M tokens calculation:

```
# Example: 7B model FP16 on A100
tokens_per_sec = 100  # Memory bandwidth limited
gpu_cost_per_hour = 3.20
tokens_per_hour = 100 * 3600 = 360,000

cost_per_million_tokens = (gpu_cost_per_hour / tokens_per_hour) * 1_000_000
                        = ($3.20 / 360,000) * 1,000,000
                        = $8.89 per million tokens
```

GPU Cost: Self-Hosting Economics (cont.)

When does self-hosting make sense?

```
Break-even volume:
API cost: $0.50 per 1M input tokens (GPT-3.5 Turbo)
Self-host cost: $8.89 per 1M tokens (including GPU)

Break-even when API cost exceeds GPU cost + ops overhead
Typically at >50-100M tokens/month
```

See: *LLM Inference Economics*

Optimization ROI Analysis

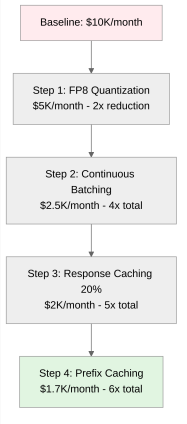
Real-world cost reduction example:

Baseline: Unoptimized serving

- 7B model, FP16, no batching, no caching
- Single A100 80GB: $4.50/hour
- Throughput: 50 tokens/sec (underutilized)
- Cost: $10,000/month for 100M tokens

Optimization ROI Analysis (cont.)

Optimization Stack:



Optimization ROI: Investment vs Savings

Investment vs Savings:

Optimization	One-time Cost	Implementation Time	Monthly Savings
FP8 Quantization	$5K	1 week	$5K
Continuous Batching	$10K	2 weeks	$2.5K additional
Response Caching	$8K	1 week	$500 additional
Prefix Caching	$2K	3 days	$300 additional
Total	**$25K**	**4-5 weeks**	**$8.3K/month**

Break-even: 3 months **Annual ROI:** 300%+

Key insight: Optimizations compound—each builds on previous

See: *LLM Inference Economics*

The Inference Cost Equation

Master formula for LLM serving costs:

```
Cost per Request = (Tokens × Time_per_Token × GPU_Cost_per_Hour) / Batch_Size
                   × (1 - Cache_Hit_Rate)
```

The Inference Cost Equation (cont.)

Breaking it down:

Variable	Impact on Cost	How to Optimize
Tokens	Linear	Limit max_tokens, truncate prompts
Time_per_Token	Linear	Quantization, better GPU, reduce model size
GPU_Cost_per_Hour	Linear	Reserved instances, spot, cheaper GPUs
Batch_Size	Inverse (÷)	Continuous batching, increase capacity
Cache_Hit_Rate	Inverse (1-x)	Better caching strategy, more cache memory

The Inference Cost Equation: Worked Example

Worked example: Serving 100M tokens/day

```
# Baseline
tokens = 100_000_000
time_per_token = 0.01  # 10ms (100 tokens/sec)
gpu_cost_per_hour = 4.50  # A100 80GB
batch_size = 1  # No batching!
cache_hit_rate = 0.0  # No caching!

daily_cost = (tokens * time_per_token * gpu_cost_per_hour / 3600) / batch_size
           = (100M * 0.01 * 4.50 / 3600) / 1
           = $1,250/day = $37,500/month
```

The Inference Cost Equation: Worked Example

Worked example: Serving 100M tokens/day

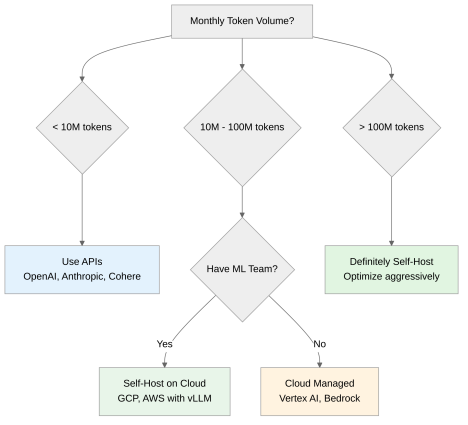
```
# Optimized
batch_size = 8  # Continuous batching
cache_hit_rate = 0.25  # 25% cache hits

daily_cost = (tokens * time_per_token * gpu_cost_per_hour / 3600) / batch_size
↪  * (1 - cache_hit_rate)
          = (100M * 0.01 * 4.50 / 3600) / 8 * 0.75
          = $117/day = $3,500/month

Savings: 10.7x
```

Takeaway: Every variable in the equation is an optimization opportunity

See: *LLM Inference Economics*

Build vs Buy Decision Matrix

Decision matrix by monthly token volume:



Build vs Buy: TCO Comparison

Total Cost of Ownership (TCO) comparison:

Approach	Upfront	Monthly Ops	Engineering	Best For
APIs	$0	$0.001-0.05/1K tokens	Minimal	Prototypes, <10M tokens
Cloud Managed	$0-5K	GPU + margin	Low	10-100M tokens, small team
Self-Hosted	$20-50K	GPU only	High	>100M tokens, ML team
On-Premise	$100K+	Electricity + maint	Very high	Data residency needs

See: *LLM Inference Economics*

Hidden Costs and Break-Even Analysis

Hidden costs of self-hosting:

- **DevOps:** Deployment, monitoring, on-call ($100K-200K/year labor)
- **Experimentation:** Testing configurations, A/B tests
- **Upgrades:** Model updates, framework updates
- **Disaster recovery:** Backups, multi-region

Hidden Costs and Break-Even Analysis (cont.)

Break-even calculation:

```python
def should_self_host(monthly_tokens_millions: float,
                     ml_team_size: int) -> bool:
    api_cost = monthly_tokens_millions * 0.50  # GPT-3.5 pricing

    # Self-hosting
    gpu_cost = 3000  # Rough estimate
    ops_cost = 10000 if ml_team_size > 0 else 20000
    self_host_cost = gpu_cost + ops_cost

    return api_cost > self_host_cost * 1.5  # 50% margin
```

Recommendation: Start with APIs, migrate when economics favor self-hosting

See: *LLM Inference Economics*

GPU Memory Bandwidth Bottleneck

During decode phase, each token requires loading entire model:

GPU	Memory Bandwidth	Tokens/sec (7B FP16)
A100 40GB	1.5 TB/s	~100
A100 80GB	2.0 TB/s	~130
H100	3.35 TB/s	~220

Optimization strategies:

1. **Quantization** → smaller weights to load
2. **Batching** → amortize weight loading across requests
3. **Better GPUs** → more memory bandwidth

See: *LLM Inference Economics*

The Idle GPU Problem

Problem: GPUs cost money even when idle

Example cost breakdown (A100 40GB):

Scenario	GPU Utilization	Hourly Cost	Waste
No batching	20%	$3.00	$2.40/hr wasted
Batching (8x)	80%	$3.00	$0.60/hr wasted

Why batching matters:

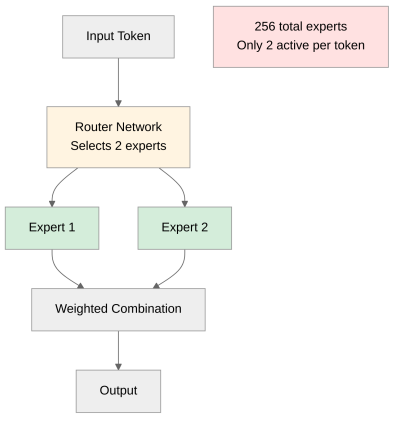
- Same hardware cost
- 4x better utilization
- 4x more requests served
- **Lower $/token** for customers

See: *LLM Inference Economics*

Case Study: DeepSeek Efficiency (2025)

DeepSeek-V3: 671B parameters, only 37B active per token

How? Mixture-of-Experts (MoE) architecture



Case Study: DeepSeek Efficiency (2025) (cont.)

Impact:

- Training cost: ~$6M (vs $100M+ typical)
- Inference: 37B active = similar speed to 37B dense model
- Capacity: 671B total = better quality than 37B

Key lesson: Architecture innovation can dramatically reduce costs

See: *Mixture-of-Experts Intro*

Advanced Serving Patterns

Model Selection Beyond Accuracy

Critical factors for production model selection:

Criterion	7B Model	13B Model	70B Model
Inference cost/token	$0.0001	$0.0002	$0.001
Memory footprint	14 GB (FP16)	26 GB	140 GB
Max concurrent (A100 80GB)	20-30 requests	10-15 requests	2-3 requests
Latency (typical)	50-100ms	80-150ms	200-400ms
Context window	4K-32K tokens	4K-32K tokens	4K-32K tokens
License	Often open (LLaMA 2)	Often open	Mixed

Model Selection Beyond Accuracy (cont.)

Decision matrix:

- **7B models:** Simple queries, high volume, cost-sensitive
- **13B models:** Balance of quality and cost
- **70B models:** Complex reasoning, quality-critical applications

Model Cascade Pattern

```python
def route_request(prompt: str, complexity: str) -> str:
    if complexity == "simple":
        return generate_7b(prompt)  # 80% of traffic
    elif complexity == "medium":
        return generate_13b(prompt)  # 15% of traffic
    else:
        return generate_70b(prompt)  # 5% of traffic
```

Complexity detection: Prompt length, keywords, user tier, task type

See: *LLM Inference Economics*

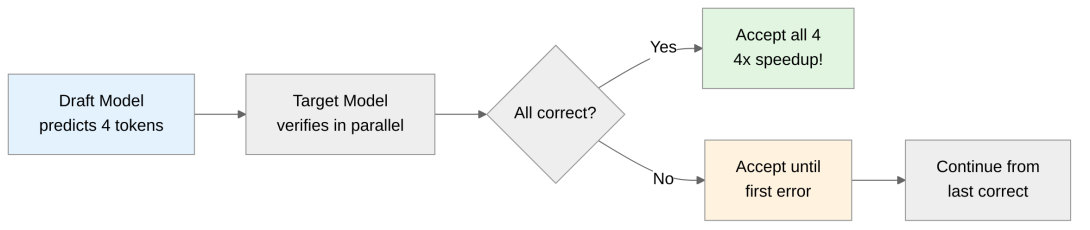
Speculative Decoding

Note: Emerging technique - See vLLM documentation for implementation details

Problem: Autoregressive decoding is sequential $\rightarrow$ slow

Solution: Use small "draft" model to predict multiple tokens, large model verifies

How it works:



Speculative Decoding (cont.)

When it works:

- Coherent, predictable text (documentation, code)
- Draft model can predict 60%+ correctly
- **Speedup: 2-3x** in ideal cases

When it fails:

- Creative writing (low predictability)
- Draft model accuracy <40%
- Overhead exceeds benefit

Implementation: Available in vLLM, TensorRT-LLM (experimental)

Prefix Caching and Sharing

Use case: Many requests share the same system prompt

Example:

```
System: "You are a helpful coding assistant..."
User 1: "How do I reverse a list in Python?"
User 2: "Explain list comprehensions"
User 3: "What are decorators?"
```

All three share the same system prompt $\rightarrow$ Cache its KV cache!

Prefix Caching and Sharing (cont.)

Memory savings:

```
# Without prefix caching
Total KV cache = 3 requests × (system + user)
               = 3 × (200 + 50) = 750 tokens

# With prefix caching
Total KV cache = 1 × system + 3 × user
               = 200 + (3 × 50) = 350 tokens

Savings: 53%
```

Prefix Caching and Sharing (cont.)

Benefits:

- Up to 50% memory reduction for chatbots
- More concurrent requests on same GPU
- Faster prefill for cached prefix

Requirements:

- Prefix must be **identical tokens** (not just similar text)
- System must track prefix reuse

Implementation: vLLM automatic prefix caching (v0.4.0+)

See: *Dynamic Request Batching*

Request Routing and Load Balancing

Challenge: Requests have varying compute requirements

Strategy 1: Round-Robin

```
Request → Next GPU in sequence
```

- **Pros:** Simple, even distribution
- **Cons:** Doesn't account for GPU load or request complexity

Strategy 2: Least-Loaded GPU

```
Request → GPU with lowest current batch size
```

- **Pros:** Better load balancing
- **Cons:** Requires shared state, potential hot spots

Request Routing: Content-Aware Strategy

Strategy 3: Content-Aware Routing

```
def route_request(prompt: str) -> int:
    prompt_length = len(tokenizer.encode(prompt))

    if prompt_length < 512:
        # Short prompts → GPU optimized for high throughput
        return gpu_pool_fast
    else:
        # Long prompts → GPU with more memory
        return gpu_pool_large_context
```

- **Pros:** Optimal resource utilization
- **Cons:** More complex routing logic

Request Routing: Content-Aware Strategy (cont.)

Implementation considerations:

- Centralized router (load balancer) vs distributed (each server)
- Health checks and failover
- Monitoring: Track utilization per GPU

Production pattern: Combine least-loaded + content-aware

See: *FastAPI LLM Server Structure*

Graceful Degradation Under Load

When system is overloaded, you have choices:

Option 1: Queue Requests

```
if current_load > capacity:
    queue.add(request)
    return {"status": "queued", "estimated_wait": "30s"}
```

- **Pros:** No requests rejected, eventual consistency
- **Cons:** Increases latency for all users

Option 2: Reject New Requests (Circuit Breaker)

```
if current_load > capacity:
    return {"error": "Service at capacity", "retry_after": 60}
```

- **Pros:** Maintains SLA for accepted requests
- **Cons:** Poor user experience for rejected requests

Graceful Degradation Under Load (cont.)

Option 3: Route to Faster Model

```
if current_load > capacity:
    # Fallback from 70B to 7B
    return generate_small_model(prompt)
```

- **Pros:** All requests served
- **Cons:** Quality degradation

Option 4: Reduce Response Length

```
if current_load > capacity:
    max_tokens = 50  # Instead of 200
    return generate(prompt, max_tokens=max_tokens)
```

- **Pros:** Faster responses, more throughput
- **Cons:** Truncated responses

Graceful Degradation: Recommended Strategy

```python
def handle_overload(load_level: float):
    if load_level < 0.8:
        return "normal"
    elif load_level < 0.95:
        return "reduce_max_tokens"  # 200 → 100
    elif load_level < 1.1:
        return "queue_with_timeout"  # Max 30s wait
    else:
        return "circuit_breaker"  # Reject new
```

SLA considerations: Document degradation policy in user agreement

Hands-On Demos

MWE: Dynamic Request Batching

Purpose: Batch multiple inference requests to maximize GPU utilization

```
[Batch 1] Processing 4 requests together
  Client 0: Response to 'Query from client 0' (batch 1)
  Client 1: Response to 'Query from client 1' (batch 1)
  ...

=== Summary ===
Total requests: 10
Batches processed: 3
Average batch size: 3.3
```

Key concept: 8 requests batched = 8x efficiency gain

MWE: LLM Benchmark Suite

Purpose: Benchmark LLM inference APIs with batching and caching measurement

```
# High concurrency test
python main.py --endpoint http://localhost:8000/generate \
    --concurrency 20 --requests 200

# Cache effectiveness test
python main.py --endpoint http://localhost:8000/generate \
    --repeat-ratio 0.5 --requests 100
```

Metrics: TTFT, TPOT, throughput, cache hit rate

Real-World Case Studies

Case Study: ChatGPT Infrastructure Evolution

From launch to massive scale:

Launch Phase (Nov 2022):

- Simple API deployment
- Immediate overwhelming demand
- Frequent outages and slow responses
- Learning: Need queue management and load balancing

Early Evolution (Q1 2023):

- Implemented request queuing
- Added capacity planning and monitoring
- Started batching optimization
- Result: More stable but still capacity-constrained

ChatGPT Infrastructure Evolution (cont.)

Scaling Phase (Q2-Q3 2023):

- Adopted continuous batching (vLLM-style)
- Multi-region deployment
- Aggressive caching layer
- Result: Handle 10x more traffic

Current State (2024-2025):

- Millions of concurrent users
- $<$1 second P95 latency
- Multi-tier model routing (GPT-4 $\rightarrow$ GPT-3.5 cascade)
- Advanced optimizations: Quantization, prefix caching, speculative decoding

ChatGPT Infrastructure Evolution (cont.)

Key lessons:

1. Start simple, optimize under load
2. Caching and batching are force multipliers
3. Multi-region critical for global latency
4. Continuous monitoring enables rapid iteration

See: *LLM Serving Frameworks*

Case Study: 10x Cost Reduction Journey

Scenario: Company serving 1 billion tokens per day

Baseline (Month 0):

```
Using GPT-4 API at $0.03/1K input + $0.06/1K output
Average: 100 input + 200 output tokens per request
Cost: 1B × (0.1M input × $0.03 + 0.2M output × $0.06) / 1M
    = $3,000 + $12,000 = $15,000/day = $450K/month
```

Step 1: Self-host with vLLM (Month 1)

- Deploy LLaMA 2 70B on 8× A100 GPUs
- GPU cost: $3/hour × 8 GPUs × 730 hours = $17,520/month
- Engineering: $50K one-time setup
- **Result: $17.5K/month (26x reduction in operating cost)**

Case Study: 10x Cost Reduction (cont.)

Step 2: Add FP8 Quantization (Month 2)

- Reduce to 4× A100 GPUs (same throughput)
- GPU cost: \$3/hour × 4 GPUs × 730 hours = \$8,760/month
- **Result: \$8.8K/month (51x reduction total)**

Step 3: Implement Caching (Month 3)

- 30% cache hit rate for repeated queries
- Effective tokens: 700M/day (30% saved)
- GPU cost: \$8,760 × 0.7 = \$6,132/month
- Redis: \$500/month
- **Result: \$6.6K/month (68x reduction total)**

Case Study: 10x Cost Reduction (cont.)

Step 4: Model Cascade (Month 4)

- Route simple queries to 7B model (40% of traffic)
- 7B runs on 1× A100, 70B on 3× A100
- GPU cost: $(1 + 3) \times \$3/\text{hour} \times 730 \times 0.7 = \$6{,}132/\text{month}$
- **Result: $6.6K/month (68x reduction sustained)**

Total Investment vs Savings:

- One-time: $50K engineering
- Monthly: $6.6K vs $450K baseline
- Break-even: ~1.1 months
- Annual savings: $5.3M

Case Study: 10x Cost Reduction (cont.)

Key insights:

1. Self-hosting justified at >50M tokens/day
2. Quantization nearly free performance drop
3. Caching effectiveness varies by use case
4. Model cascade requires routing logic but worth it

Debugging a Latency Spike: Investigation

Problem Report:

```
Alert: P95 latency spiked from 500ms to 3+ seconds
Time: 2:47 PM on production cluster
Impact: 5% of requests affected
User complaints: "System is slow/frozen"
```

Debugging a Latency Spike: Investigation (cont.)

Investigation Process:

Step 1: Check Infrastructure (2:50 PM)

```
# GPU utilization
nvidia-smi
# Result: 75-80% utilization (normal range)

# Memory usage
# Result: No OOM errors, memory at 65%

# Conclusion: Not an infrastructure issue
```

Debugging a Latency Spike: Investigation (cont.)

Step 2: Analyze Request Metrics (2:55 PM)

```
# Check batch size distribution
SELECT percentile_95(batch_size) FROM metrics
# Normal: 4-8, Current: 28-32 (ANOMALY!)

# Check request distribution
SELECT percentile_95(prompt_length) FROM metrics
# Normal: 512 tokens, Current: 8192 tokens (ANOMALY!)
```

Debugging a Latency Spike: Investigation (cont.)

Step 3: Identify Root Cause (3:00 PM)

- Batch sizes abnormally large $\rightarrow$ Processing taking longer
- Few requests with 8K+ token prompts
- No maximum prompt length enforcement
- These long requests were blocking entire batches

Debugging a Latency Spike: Resolution (cont.)

Step 4: Implement Fix (3:15 PM)

```python
# Add prompt length limit
MAX_PROMPT_LENGTH = 2048

def process_request(prompt: str):
    if len(tokenizer.encode(prompt)) > MAX_PROMPT_LENGTH:
        # Truncate or reject
        prompt = truncate_prompt(prompt, MAX_PROMPT_LENGTH)
    return generate(prompt)
```

Step 5: Verify Resolution (3:30 PM)

```
P95 latency: Back to 550ms
Batch sizes: Normal 4-8 range
User complaints: Resolved
```

Debugging a Latency Spike: Resolution (cont.)

Lessons Learned:

1. **Always have limits:** Max prompt length, max batch size, request timeouts
2. **Monitor distributions:** P95/P99 catch tail latencies
3. **Systematic debugging:** Check infrastructure $\rightarrow$ metrics $\rightarrow$ code
4. **Quick mitigation:** Implement safeguards even if not perfect

Prevention for Future:

- Add validation layer: Reject prompts $>$4K tokens
- Implement request timeouts (10s per request)
- Better monitoring: Alert on batch size anomalies
- Load testing: Include long-prompt scenarios

Lab & Assessments

Lab: Dynamic Request Batching

Objectives:

- Implement a dynamic request batcher using asyncio
- Use `asyncio.Lock` to prevent race conditions
- Configure batch size and timeout parameters
- Measure batched vs. unbatched performance

Expected Results:

- Single request: ~55ms
- 8 requests batched: ~90ms total (~11ms each)
- **Speedup: 4-5x faster with batching!**

Milestone 5: LLM Inference Optimization

Deliverables:

1. **LLM Inference Server** - Dynamic batching + caching layer
2. **Benchmark Suite** - Latency, throughput, cache metrics
3. **Performance Report** - 3-4 pages with trade-off analysis
4. **Governance Memo** - 1 page on privacy and retention

Rubric (10 pts): Batching (3) + Caching (3) + Benchmarking (2) + Governance (1) + Code Quality (1)

Homework Assignments

HW 6.1: Quantization Trade-offs

- Compare FP16, INT8, INT4 model variants
- Measure memory, latency, quality metrics
- Technical brief on quantization mechanisms

HW 6.2: Stress Testing

- Load test at multiple tiers (10, 50, 100+ req/s)
- Identify saturation points and bottlenecks
- Propose data-driven autoscaling policies

HW 6.3: Inference Router

- Route requests based on model/load characteristics

Deployment and Future Trends

Deployment Pattern Selection

Four main deployment approaches for LLM serving:

Pattern	Best For	Pros	Cons
Managed API	<10M tokens/month	No ops, easy start	Higher cost, API lock-in
Cloud Managed	10-100M tokens/month	Balanced control	Platform-specific
Self-Hosted	>100M tokens/month	Full control, cost-effective	Requires ML ops expertise
On-Premise	Compliance needs	Data residency	High capital cost

Deployment Pattern Selection (cont.)

Decision framework:

- Low volume + small team → Use APIs (OpenAI, Anthropic)
- Medium volume + some ML expertise → Cloud-hosted (GCP Vertex AI, AWS Bedrock)
- High volume + ML team → Self-host with vLLM on cloud VMs
- Strict compliance → On-premise with air-gapped infrastructure

See: *LLM Inference Economics*

Emerging Optimization Techniques

What's coming in 2026-2027:

Flash Attention 3:

- Further memory optimization for long contexts
- Reduced memory bandwidth requirements
- Faster prefill phase

Mixture-of-Depths:

- Activate subset of layers per token (not just experts)
- Dynamic depth allocation based on token complexity
- Potential 2-3x speedup for simple tokens

Emerging Optimization Techniques (cont.)

Knowledge Distillation:

- Train smaller models from larger teachers
- Preserve quality while reducing inference cost
- Example: 70B teacher $\rightarrow$ 7B student with 90% quality

Model Merging:

- Combine specialized models for different tasks
- Weight averaging and task routing
- Build custom models without retraining

What to watch: Research papers from NeurIPS, ICML, and arXiv

Infrastructure Evolution

GPU roadmap:

- H100 (current, 2024-2025): 3.35 TB/s memory bandwidth
- B100 (2025-2026): Expected 5+ TB/s, improved FP8 support
- Blackwell architecture (2026+): Further efficiency gains

Alternative accelerators:

- AMD MI300X: Competitive pricing, 5.3 TB/s bandwidth
- Google TPU v5: Optimized for Google Cloud workloads
- AWS Trainium: Custom silicon for AWS customers
- Groq LPU: Ultra-low latency specialized chips

Infrastructure Evolution (cont.)

Edge deployment trends:

- Quantized models (INT4) running on mobile devices
- On-device inference for privacy-sensitive applications
- Federated learning for model updates

Serverless evolution:

- Pay-per-token pricing models
- Auto-scaling to zero
- Cold start optimizations ($<$1 second)

Cost trajectory: Expect continued 2-3x annual efficiency improvements

See: *LLM Inference Economics*

Key Takeaways

Module Summary

1. **LLM architecture affects serving**: Autoregressive generation, KV cache, prefill vs. decode phases all impact optimization strategies
2. **Batching is essential**: Amortizing weight loading across requests can provide 5-8x throughput improvement
3. **Caching reduces costs**: 20-40% cache hit rates can cut infrastructure costs by 30-50%
4. **Governance is critical**: Privacy-preserving cache design, retention policies, and rate limiting are production requirements

Next Module Preview

Module 7: LLMOps II - RAG and Agentic Systems

- Retrieval-Augmented Generation (RAG) architecture
- Vector databases and embedding models
- Document chunking and retrieval strategies
- Agentic AI patterns and multi-tool coordination